Informe de pruebas y cobertura - SkinMarket Pro

Evaluación Parcial 2 - DSY1104 Desarrollo Fullstack II. Fecha: 19 de octubre de 2026.

1. Objetivo y alcance

Verificar las reglas de negocio y el comportamiento de los componentes React del prototipo: registro, login, filtros, carrito, compras, inventario y administración. La suite contiene 29 pruebas en seis archivos e incluye pruebas unitarias y pruebas de integración de componentes; no se presentan todas como pruebas unitarias aisladas.

La persistencia es localStorage. El login, los roles, el saldo y las compras son simulaciones de frontend. No existe autenticación de servidor, pago bancario ni sincronización entre computadores. Las pruebas no acreditan seguridad productiva ni integración con un backend.

2. Entorno y herramientas

- Jasmine 7.0.2: suites, expectativas, matchers y spies.

- Karma 6.4.4: ejecuta las pruebas en Google Chrome Headless; ejecución verificada en Chrome 155 sobre Windows.

- Testing Library React y user-event: montaje e interacción con componentes reales, usando consultas por rol y etiquetas.

- Webpack y Babel: transforman imports y JSX para el navegador. No reemplazan Vite en la aplicación.

- babel-plugin-istanbul y karma-coverage: instrumentación y reportes HTML, JSON y resumen de terminal.

- Bootstrap 5.3.8: columnas responsivas del catálogo y acceso; controles y errores del formulario de acceso. CSS propio mantiene la temática visual.

`karma.conf.cjs` define el entorno, el navegador, la transformación JSX, los recursos de imágenes y el mínimo de cobertura. `tests/entry.js` carga todos los módulos evaluables y los seis archivos spec.

3. Procedimiento reproducible

Requisitos: Node.js 22.18+ (rama 22) o 24.11+ y Google Chrome instalado. Ejecución verificada con Node 24.13.1. Si Chrome no se detecta automáticamente, configurar CHROME_BIN con la ruta de su ejecutable.

```
npm ci
npm run test:coverage
npm run build
```

Karma inicia un servidor local, abre Chrome Headless, ejecuta Jasmine y termina devolviendo un código de error si falla una prueba o un umbral de cobertura. `npm run test:watch` mantiene la ejecución activa durante el desarrollo. Los reportes quedan en `coverage/index.html` y `coverage/coverage-summary.json`.

4. Catálogo de pruebas

Archivo	Casos	Verificaciones principales
tests/market.spec.js	7	Correo, nombre y contraseña; precio/stock; persistencia y corrupción; filtros y orden; moneda e imágenes; compra atómica y rechazos.
tests/context.spec.jsx	4	Login, restauración y logout; registro único y permisos; cantidades, saldo y compras; CRUD y restauración de demostración.
tests/app.spec.jsx	10	Inicio, búsqueda y navegación; login y recarga; registro; inventario e historial; carrito y errores; rutas protegidas; cliente sin acceso admin; 404; CRUD visual.
tests/admin.spec.jsx	6	Estadísticas; validaciones y administrador activo; bloqueo y sincronización; gráficos vacíos; ventas agregadas; búsqueda, edición y bloqueo de usuarios.
tests/modal.spec.jsx	1	Foco inicial, ciclo Tab/Shift+Tab, Escape, cierre por fondo y devolución del foco.
tests/id.spec.js	1	UUID nativo y alternativa mediante getRandomValues para contextos sin randomUUID.
Total	29	Reglas puras, contexto y componentes integrados.

5. Diseño de casos representativos

1. Registro válido: nombre Alex, correo válido y contraseña de seis caracteres. Se crea una cuenta cliente, sin privilegios administrativos.

2. Registro inválido: nombre vacío, correo incorrecto o contraseña corta. Se devuelven errores por campo y no se registra la cuenta.

3. Correo duplicado: se intenta reutilizar un correo existente. El registro se rechaza.

4. Login: se comprueban correo y contraseña, credenciales incorrectas, restauración de sesión y cierre de sesión.

5. Precio y stock: se rechazan negativos, texto, infinito y stock fraccionario; se aceptan ceros válidos.

6. Compra correcta: se descuenta saldo y stock, se vacía el carrito y se conserva un pedido y una entrada de inventario sin mutar el estado original.

7. Compra rechazada: sesión inexistente, carrito vacío, saldo insuficiente, producto eliminado o cantidad inválida. No se aplica una compra parcial.

8. Filtros: se combinan texto, categoría, rareza, colección, exterior y precio; máximo cero se respeta y ordenar no altera el arreglo original.

9. Administración: se crea, modifica y elimina un producto con confirmación; un cliente no puede ejecutar esas operaciones.

10. Usuarios y accesibilidad: bloquear impide login/compra sin borrar el historial; no se permite autobloqueo del administrador. Un modal devuelve el foco al cerrar.

Estos ejemplos no sustituyen el catálogo completo; los archivos spec contienen las entradas y expectativas ejecutables.

6. Aislamiento, mocks y aserciones

Antes de cada prueba se limpia localStorage y se registran matchers de Jasmine. Después se desmontan los componentes con cleanup. Cada escenario construye sus propios datos mediante freshState; las pruebas no modifican las compras del sitio publicado, porque Karma usa otro origen local.

`spyOn(window, 'scrollTo').and.stub()` evita desplazamientos ajenos al objetivo de las pruebas. `jasmine.createSpy('onClose')` registra cuántas veces se solicita cerrar un modal. Un doble de almacenamiento que lanza errores en getItem/setItem permite verificar recuperación y rechazo de escritura. La prueba de UUID inyecta un proveedor criptográfico determinista para comparar el resultado.

Se conservan React, Context, localStorage del navegador y las interacciones de usuario; no se reemplaza toda la aplicación por mocks. Las aserciones comprueban resultados, mensajes, permisos y estado persistido. `toThrowError` comprueba rechazos; los matchers DOM registrados con jasmine.addMatchers comprueban foco, texto, atributos y controles deshabilitados.

Ejemplo real: tests/modal.spec.jsx crea un spy onClose, simula Escape y comprueba `expect(close).toHaveBeenCalledTimes(1)`.

7. Resultados y cobertura

Ejecución verificada: 29 aprobadas, 0 fallidas. Compilación de producción correcta mediante npm run build.

Métrica	Cubierto / total	Resultado	Mínimo configurado
Sentencias	550 / 573	95,98 %	60 %
Ramas	532 / 555	95,85 %	60 %
Funciones	223 / 234	95,29 %	60 %
Líneas	492 / 513	95,90 %	60 %

Se instrumenta src/**/*.{js,jsx}, incluidas páginas, componentes, contexto, rutas y reglas. Se excluye src/main.jsx por ser el montaje de React y src/data/** por contener datos estáticos; CSS, imágenes y dependencias no forman parte de esta cobertura. La carga explícita de módulos evita ocultar archivos sin pruebas por no haber sido importados.

La cobertura indica qué código se ejecutó, no demuestra que todos los escenarios posibles estén correctos. Quedan 21 líneas, 23 sentencias, 23 ramas y 11 funciones sin cubrir. El reporte HTML permite localizar esos huecos. El umbral del 60 % es el configurado para este proyecto, no una equivalencia automática con una nota.

8. Riesgos, límites y revisión complementaria

- La suite no mide rendimiento, concurrencia real, accesibilidad completa, seguridad de servidor ni una pasarela de pago.

- Revisión visual completada en Chrome: escritorio de 1536 px, móvil de 390 px y tableta de 768 px. Se comprobaron portada, columnas del catálogo, acceso móvil, menú y filtros móviles, y editor administrativo. No se observó desbordamiento horizontal de la página en esos tamaños. Las categorías conservan su desplazamiento horizontal interno intencional. No equivale a probar todos los dispositivos ni todos los navegadores.

- npm audit detecta 9 avisos en dependencias transitivas de desarrollo: 5 moderados y 4 altos. npm audit --omit=dev no detecta vulnerabilidades. No se usó --force porque propone cambios incompatibles; esto no equivale a eliminar el riesgo del entorno de desarrollo.

- S3 actualizado con index.html y los dos nuevos recursos compilados; consola confirmó tres archivos correctos y cero errores. Recursos de esta versión: assets/index-Cu_TRYIU.js y assets/index-B8lAfug-.css. Se retiró el bloque informativo Juego / Counter-Strike 2 de los filtros; la prueba de catálogo comprueba su ausencia y la permanencia del selector Tipo de arma. No se modificaron permisos ni se borraron recursos antiguos. Compilar y subir a GitHub no publican automáticamente futuras versiones.

9. Evidencias y entrega

Código de pruebas en tests/, configuración en karma.conf.cjs y dependencias en package-lock.json. El informe PDF acompaña al proyecto; el ZIP de entrega incluye el reporte HTML/JSON de cobertura y la salida de pruebas para revisión sin regenerar. No incluye node_modules ni credenciales de AWS.

Conclusión: la ejecución automatizada respalda el funcionamiento probado y supera los cuatro umbrales de cobertura. La revisión visual y la actualización de S3 también fueron comprobadas por separado. No se acredita un backend productivo ni se garantiza una calificación: el informe documenta el alcance real de las verificaciones.